

Designing an AI to play Minesweeper using Reinforcement Learning

Haesung Becker, Jared Lyon

Fall 2025

CS 271

Summary

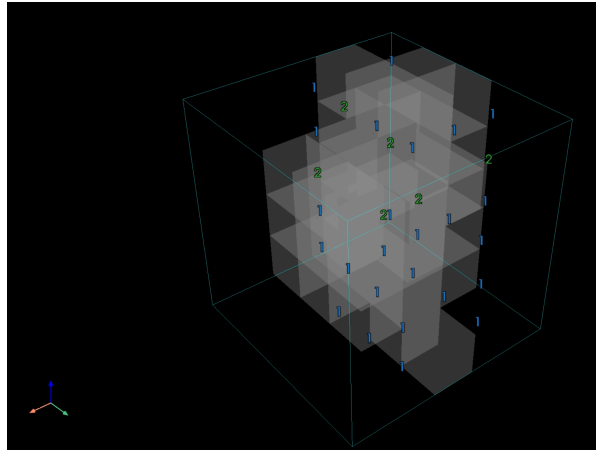
The primary goal of this project was to successfully implement a reinforcement learning-based agent to play and solve 3-dimensional Minesweeper. Originally, our project proposal timeline stated a preliminary implementation of such an agent within the original 2D game space with an intention to move on to a 3D implementation at a later point. We decided to forego the 2D implementation entirely because we did not believe the characteristics of the state space to properly motivate the usage of reinforcement learning, as the majority of 2D minesweeper games can be solved with raw boolean logic.

The introduction of a third dimension into the Minesweeper game state motivates the usage of reinforcement learning because, unlike its 2D counterpart, there is no widely agreed upon strategy to win. Therefore, we hoped the exploratory nature of reinforcement learning algorithms would prove instrumental in uncovering a reliable method through which to approach the state space. Specifically, there are two primary reasons as to why 3D Minesweeper is exceedingly more complicated. First, the expansion of each tile's total number of possible neighbors expands from 8 to 26, causing the probability of mine clusters to increase:

$$P_{\text{adjacent mine 2D}} \approx 1 - (1 - p)^8 < P_{\text{adjacent mine 3D}} \approx 1 - (1 - p)^{26} \quad (1)$$

The increase in frequency of mine clusters introduces complications to the state space, as mine clusters are both logically and more computationally difficult to navigate. Second, the 3D nature of the mine clusters renders solution matrices to be exponentially more complicated than in 2D Minesweeper. For humans, this results in increased visualization complexity, whereas computational models struggle with combinatorial explosions. This is also why 3D Minesweeper is typically played on a much smaller board than 2D Minesweeper.^{1,2}

It follows that a reinforcement learning approach would likely be the most efficient implementation when attempting to solve this type of 3D state space, as a well-adapted model should find a strategic compromise to discern ‘best moves’ without necessarily requiring the hefty computational work that typically causes combinatorial runaway.



Example of Observable State Space in 3D Minesweeper

Approaches

The environment used in this project was custom built from scratch, although inspired by previous implementations of 2D environments.³ As is the norm in other 3D implementations for human players, agents were not allowed to interact with ‘buried’ tiles, and instead constrained to an observable state space consisting of tiles that were bordered by at least one empty or revealed tile. Agents were given information regarding which tiles were unrevealed, and how many mines were contained in the neighbors of these tiles.

The leading barrier that agents faced was information scarcity, particularly concerning mine clusters. In 2D Minesweeper, there exist certain configurations of mine locations which force the player to guess; consequently, not all games of 2D Minesweeper are solvable with logic alone.⁴ This problem is further exacerbated in 3D Minesweeper due to the combinatorial nature of possible solution sets, therefore achieving a perfect 100% winrate is not possible. Furthermore, there exists a lack of opportunity to diversify reward feedback since there exist only two outcomes when revealing a tile (safe tile vs. mine). In order to better assist reinforcement learning implementations, we chose to reward actions that revealed more than one tile, which was possible due to the environment’s embedded functionality of automatically recursively revealing all neighboring tiles of a chosen 0-mine tile, because they also cannot contain mines. Agents were also rewarded with a -100 or +100 score for losing or winning the game.⁵

```
# Terminal handling
if self.game.game_over:
    terminated = True
    truncated = False
    reward = 100.0 if self.game.win else -100.0
    safe_move = 1 if self.game.win else 0
    new_visible = np.sum(self.game.visible >= 0)
    safe_tiles = max(new_visible - prev_visible, 0)

    info["safe_move"] = safe_move
    info["safe_tiles"] = safe_tiles

    return obs, reward, terminated, truncated, info

# Reward for progress
new_visible = np.sum(self.game.visible >= 0)
delta = int(new_visible - prev_visible)
reward = float(delta)
safe_tiles = max(delta, 0)
```

Reward Shaping as Implemented in the Environment

We approached the problem by implementing four separate agents: a random agent, a simple heuristic agent, a Bayesian approximation agent, and a dueling convolutional neural

network. However, the random agent contains no functional code to interpret information in the observable state space, chooses tiles at random, and serves primarily as a baseline testing module to guarantee environment validity and proper communication between the agent and the state space.

The first baseline agent implemented was a simple heuristic agent which was instructed to always choose neighbors of 0 tiles, as these tiles are guaranteed to be safe moves. If there were no available 0 tiles, then this agent would choose a tile at random. Intuitively, this is not a good strategy for Minesweeper in general, but it is possible for this agent to win games where mines are far enough apart such that the only non-zero tiles are 1 tiles. In other words, the success of this heuristic agent relies on the probability of the state space rendering an easily solvable board, although the solution itself would be trivial since a human would be able to accomplish the same task quickly and easily.

The second baseline agent implemented was one that utilized Bayesian approximation.⁶ As mentioned previously, the solution matrices of 3D Minesweeper introduce incredibly computationally intensive combinatorial math which makes calculation of exact Bayesian probabilities unsustainably expensive. Preliminary iterations of this agent which used exact prior-posterior calculations caused runtime hangs that prevented the agent from finishing episodes within a reasonable period of time. As a result, we elected to use Bayesian approximation:

$$P_{mine} = \frac{1}{|R(c)|} \sum_{r \in R(c)} \frac{n_r}{u_r}, \text{ if } |R(c)| > 0 \quad (2.1)$$

where for any tile c :

$R(c)$ is the set of revealed neighboring cells of c that contain a number,

n_r is the number revealed on tile r ,

u_r is the count of unrevealed neighbors of r .

Equation 2.1 estimates the mine probability of a hidden cell by using the constraints provided by its adjacent numbered tiles. Each numbered tile indicates how many mines exist within its surrounding unrevealed neighbors, and the agent assumes these mines are uniformly distributed across those cells. This yields a local probability estimate for the candidate cell, and when multiple numbered tiles provide overlapping constraints, the agent computes the average of their estimates. This allows for non-intensive computation of locality based probability approximations for all legal moves within the state space.

$$P_{mine} = p_0, \text{ if } |R(c)| = 0 \quad (2.2)$$

In the case that the set of revealed neighbors do not have a number, the agent defaults to a baseline probability p_0 given by the number of mines in the state space divided by the total number of both revealed and unrevealed cells on the board. It follows that this Bayesian approximation agent subsequently chooses to reveal the tile which it ascertains to have the lowest probability of containing a mine, tie-broken by the ordering in which the tiles are discovered.

Finally, we implemented a dueling convolutional neural network agent to fulfill the original project goal.⁷ However, within our time and compute budget, we were not able to train a DQN agent that outperforms the raw probabilistic reasoning used by the Bayesian approximation agent.

We designed our DQN to maximize performance and stability under limited computational resources. The core model is a 3D convolutional neural network, which naturally captures local spatial structure on the voxel grid. This gives the network a strong inductive bias for the geometry of the state space and helps it learn patterns such as “mine-counting” relationships more efficiently than a fully connected architecture would. On top of the convolutional stack, we use a dueling architecture that separately estimates the state value $V(s)$ and the action advantages $A(s, a)$ before recombining them into Q-values. This separation is helpful in our setting because many actions in a given state are nearly interchangeable (e.g., clicking among multiple safe tiles), so explicitly modeling the underlying state value makes the optimization problem better conditioned.

$$Q(s, a) = V(s) + (A(s, a) - \frac{1}{|A|} \sum_{a' \in A} A(s, a')) \quad (3)$$

For training, the agent uses two key techniques to improve stability and data efficiency. First, we adopt a double DQN setup with an online network and a target network. The online network chooses the greedy next action, and the target network evaluates that action when computing the Bellman target. This decoupling reduces the maximization bias that can otherwise cause systematic overestimation of Q-values and lead to suboptimal policies in a complex, high-dimensional state space. Second, we employ action masking: tiles that are already revealed are treated as illegal actions and are explicitly masked out before selecting an action. This injects simple prior knowledge and dramatically reduces the effective action space, allowing the agent to spend its function-approximation capacity and sample budget on learning strategy rather than repeatedly exploring obviously invalid moves.

Despite these design choices, the state and action spaces still imposed significant limitations. Training required very long horizons—on the order of 10^5 episodes before the learning curve stabilized on easier configurations—and the network is not designed to generalize across board sizes, since the final fully connected layers expect a fixed input shape. Because the number of possible actions grows cubically with the board dimensions, both the size of the output layer and the sample complexity scale poorly. In practice, this means our dueling CNN agent works reasonably well on the smallest boards we tested, but is effectively constrained by

architecture rigidity and training cost as we move to larger or denser 3D Minesweeper environments.

We trained the dueling 3D CNN DQN with 9 Conv3d layers ($3 \times 3 \times 3$ kernels), a 512-unit shared fully connected head with 256-unit dueling streams, Adam optimizer ($\text{lr} = 1\text{e-}4$), replay buffer of 500k transitions, batch size 256, $\gamma = 0.99$, and an ϵ -greedy policy that linearly decays from 1.0 to 0.05 over 200k steps after a 10k-step warmup.⁸

The relatively deep 3D CNN architecture provides the capacity needed to learn complex, non-linear relationships specific to the 3D voxel grid. However, this depth also requires the usage of robust techniques like Adam optimization and a large batch size to maintain training stability and reduce gradient variance, especially when training on a GPU (our agent was trained using an NVIDIA RTX 4070). A large replay buffer capacity was used to ensure high data efficiency by forcing the model to train on uncorrelated experience sets and thus prevent catastrophic knowledge loss as the policy shifts. The discount factor assigns high importance to future rewards, as the design of Minesweeper (immediate loss on mine) creates an environment where immediate safety is less important than long-term survivability.

These design choices specialize the model for the state space but introduce severe scaling limitations. While the large ϵ -decay schedule and warmup period favor thorough exploration over initial speed and prevent final policies that fall into local optima, this is also why training required an extraordinarily large step count. As mentioned previously, the 512/256 unit dueling streams provide necessary approximation power, but it is their fixed sizing which rigidly constrains any one architecture to the atomic constraints of board size and mine count. The inability to scale to larger 3D boards in addition to the inherently high computational cost of 3D convolutions creates a hard upper limit on the complexity of environments this agent can learn on.

Evaluation

All four models were evaluated using multiple key performance metrics: reward outcomes, move count statistics, win/loss ratios, and average episodic timing. While the random, heuristic, and Bayesian agents do not make use of reward shaping, the environment was still configured to calculate reward outcomes on an episodic basis in an identical manner to how the reinforcement learning agent would have received such data. This offered a unified framework to contrast fundamentally different algorithmic approaches. Even though the baseline agents do not optimize for these numerical rewards, scoring them identically reveals whether the RL agent's "high scores" truly reflect intelligent gameplay comparable to the heuristic and Bayesian agents.

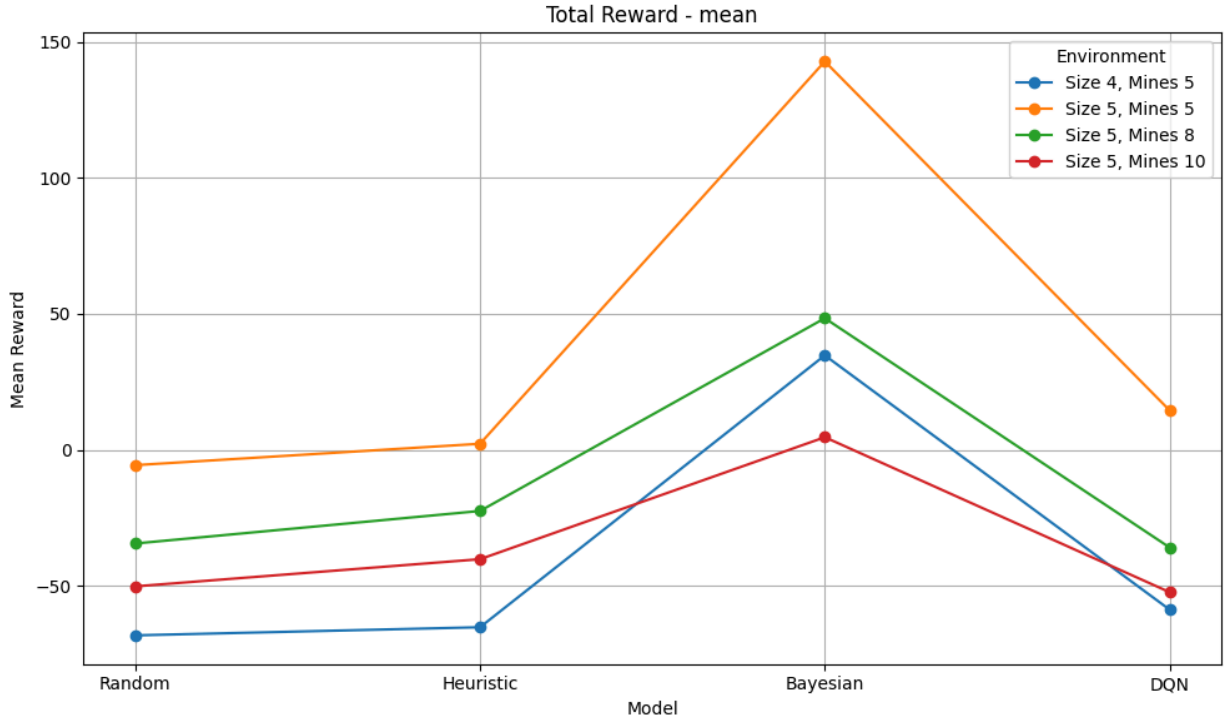


Fig. 1: Average Episodic Rewards

However, it is important to note that Figure 1 is not an accurate comparison between environments as it is for model evaluation. This is because board size and mine count directly raise or lower the available rewards granted by the board itself. Next, models were also evaluated in a similar manner to reward metrics through assessment of the average number of moves made prior to game end, regardless of win or loss.

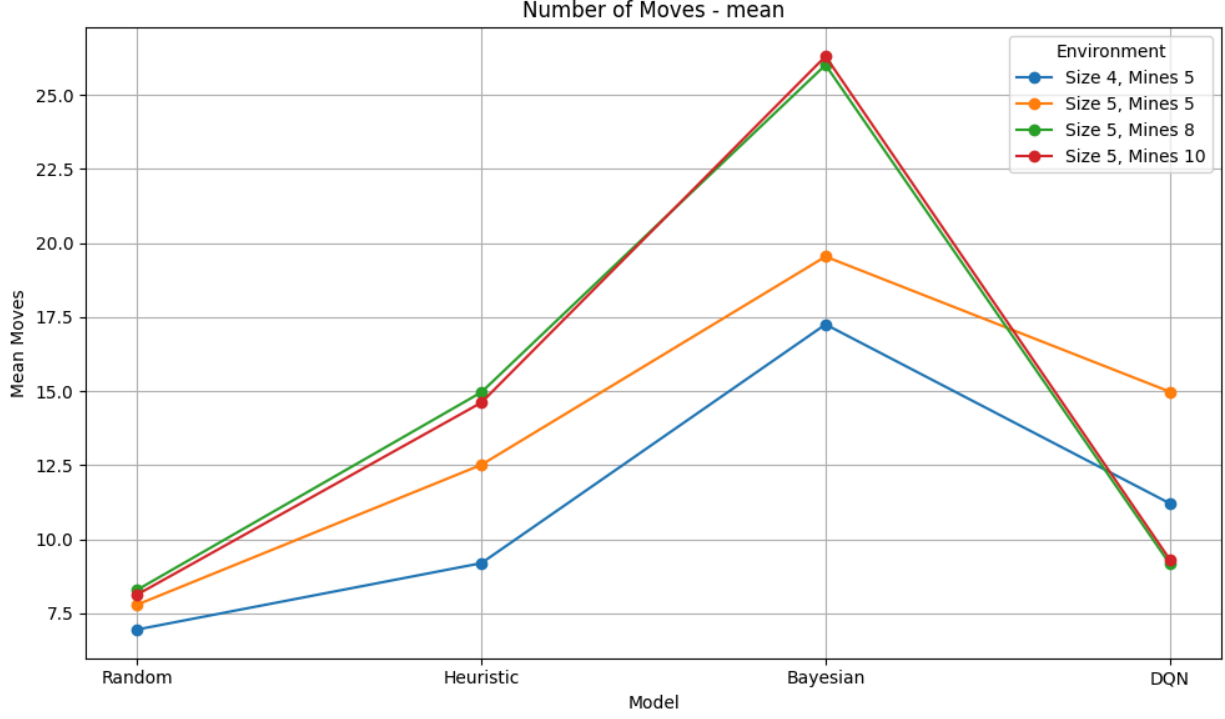


Fig. 2: Average Moves Completed

Figures 1 and 2 reveal generalized differences in performance outcomes between the models and how different environment constraints affected gameplay. It is clear from these figures that the approximate Bayesian agent vastly outperformed all other models. Review of gameplay timelines revealed that the DQN agent frequently chose to make exploratory moves within the state space when confronted with information clusters which indicated a high probability of mine presence. On the other hand, the Bayesian agent focused on sequentially choosing tiles closer together within the portion of the state space which it had already explored, therefore exhibiting its intrinsic bias towards information rather than exploration.

This behavior likely manifested as a direct result of the reward shaping in the environment, as the DQN agent would have learned that choosing tiles within the unexplored state space would have a likelihood of returning a higher reward if the environment revealed more than one tile, as opposed to a lower, although still positive, reward return had it chosen to reveal a singular tile with numbered neighbors. The tendency of the DQN agent to explore rather than exploit at a higher rate resulted in frequent mine encounters, which are reflected in the overall win rates of the models.

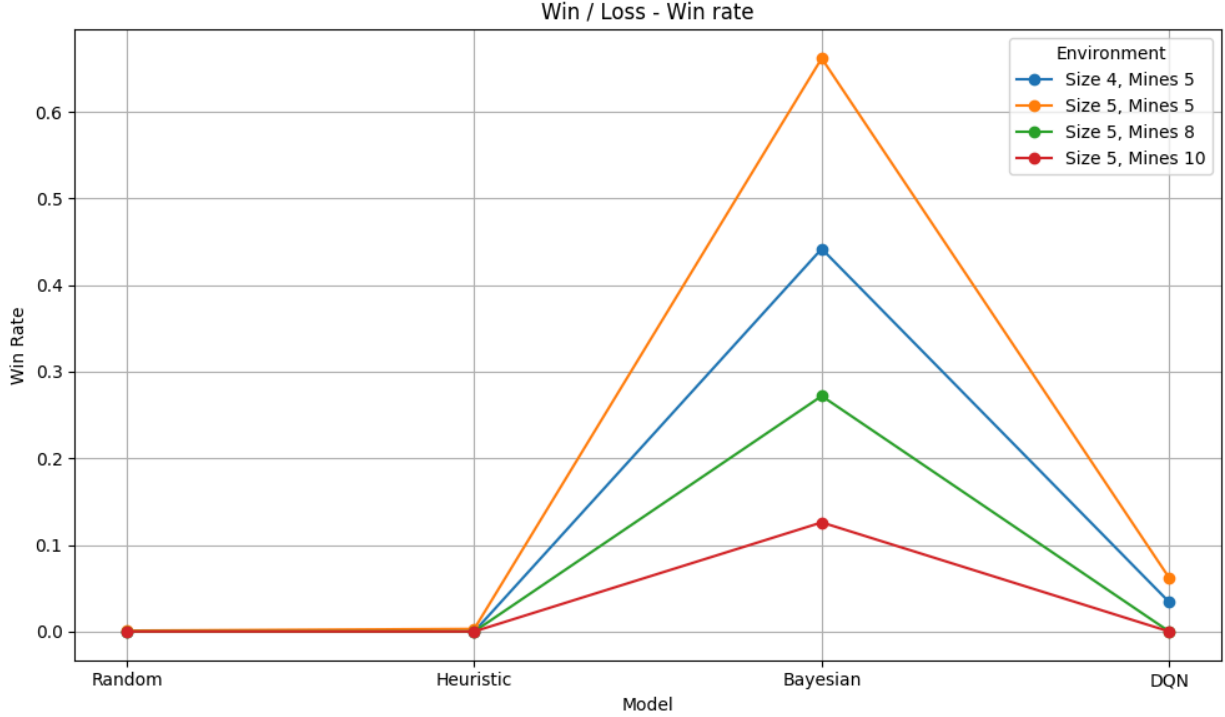


Fig. 3: Agent Win Rates

Figure 3 indicates both expected and unexpected behaviors with our preliminary hypotheses. As expected, the random and heuristic agents maintained an effective 0% win rate. The Bayesian agent maintained higher win rates overall, although its performance diminishes drastically as the environment becomes more difficult to navigate as mines increase. The DQN agent was able to win roughly ~0.5% of the time in less complicated environments (Size 4 - Mines 5, Size 5 - Mines 5), but struggled to find success in more complicated environments. This outcome is also demonstrated within the reward curve graphing in Figures 4 and 5.

Comparison of these reward curves shows that our implementation successfully converged at a near 0 reward average in the Size 5, Mines 5 environment (the Size 4, Mines 5 reward curve was similar in structure), whereas more constrained environments became unnavigable for the model. This elucidates the primary pitfall of the DQN agent, which is that with increasing mine counts the environment eventually contains too many cases of insufficient information for the reinforcement learning algorithm to properly learn how to avoid losing.

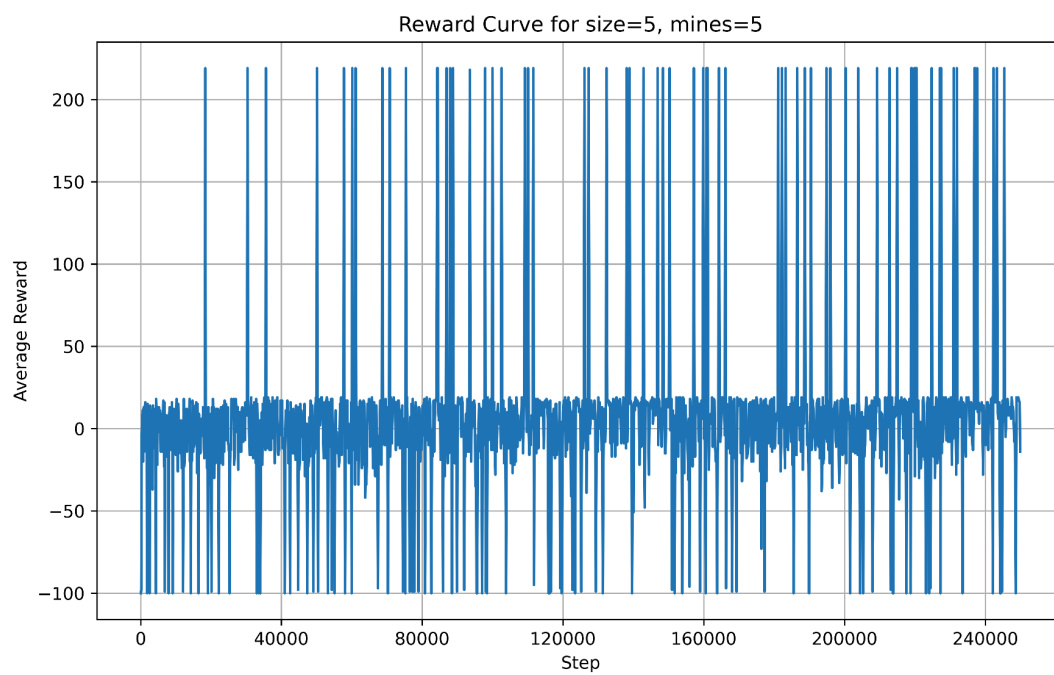


Fig. 4: DQN Reward Curve - s5, m5

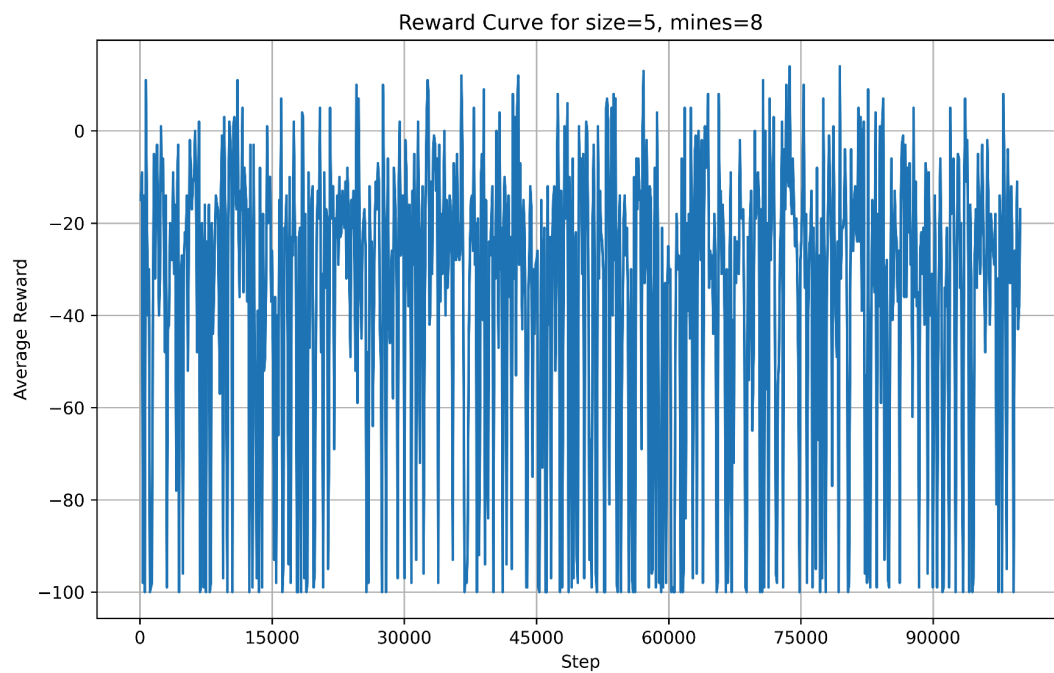


Fig. 5: DQN Reward Curve - s5, m8

Project post-mortem analysis reveals that the DQN agent was underfit to the 3D Minesweeper environment, an outcome we believe stems in large part from the absence of meaningful preliminary knowledge injection. Although 3D Minesweeper frequently forces players to guess, the state space still contains strong structural regularities that a learning system should be able to exploit. Uncovering these patterns with such sparse reward feedback thus emerges as the preliminary hurdle for this problem.

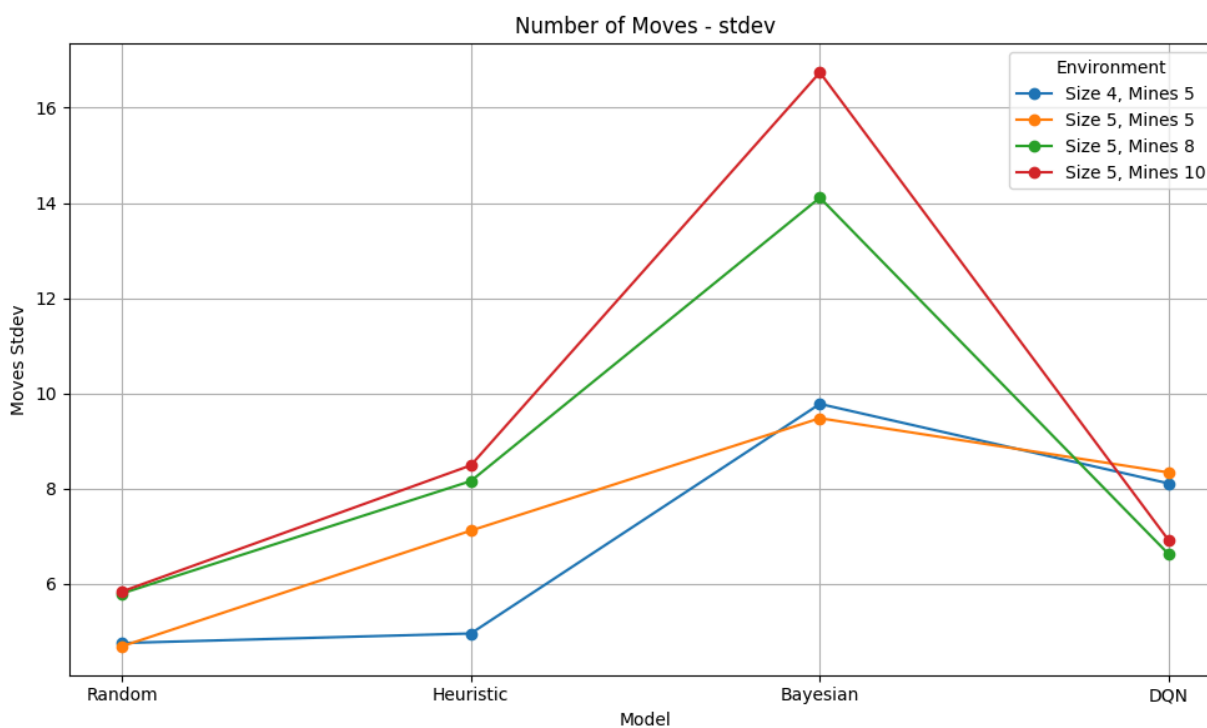
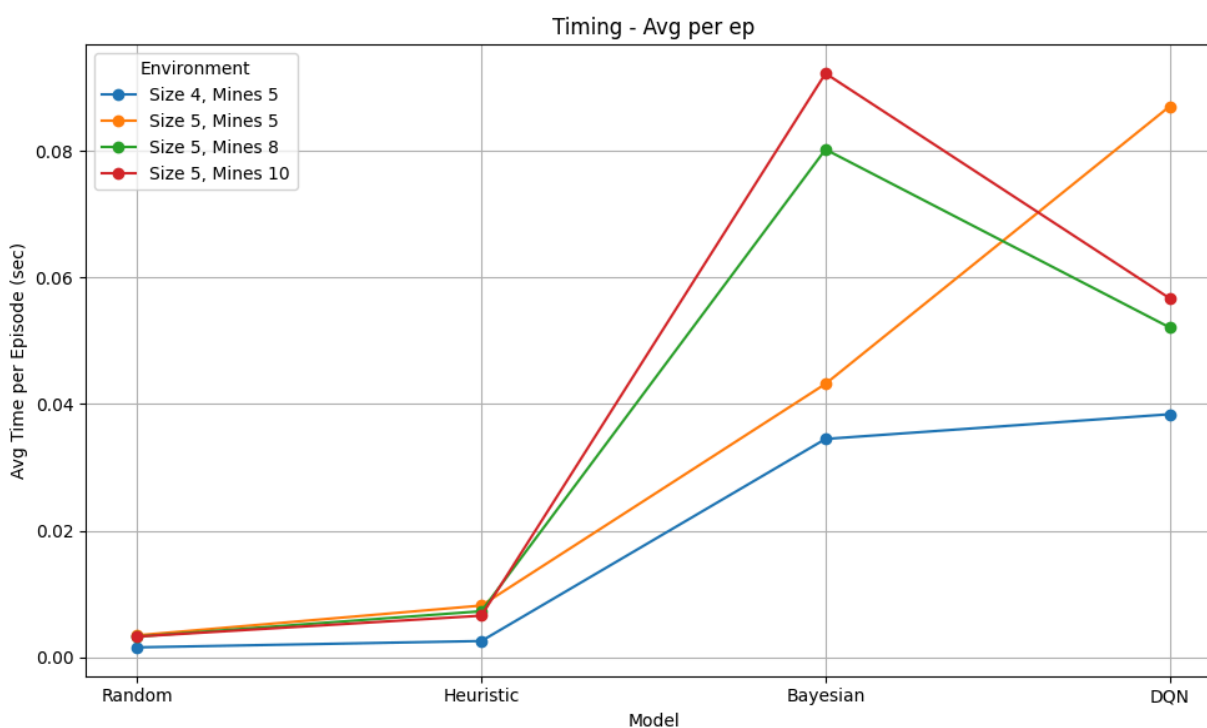
In order to circumvent this, several forms of prior knowledge could have been injected to guide the agent toward more fundamental game logic without necessarily needing high batch training. For example, the injection of logical inference rules such as “neighbors of 0 tiles are safe” or “numbered tile with N hidden neighbors and N high-probability mine neighbors indicates remaining neighbors are safe” could have been hard-coded into the state representation or incorporated as action-masking heuristics. Furthermore, encoding rotation/reflection equivariance could also allow the model to generalize across equivalent board configurations which are spatially nonequivalent, reducing overall computation complexity.

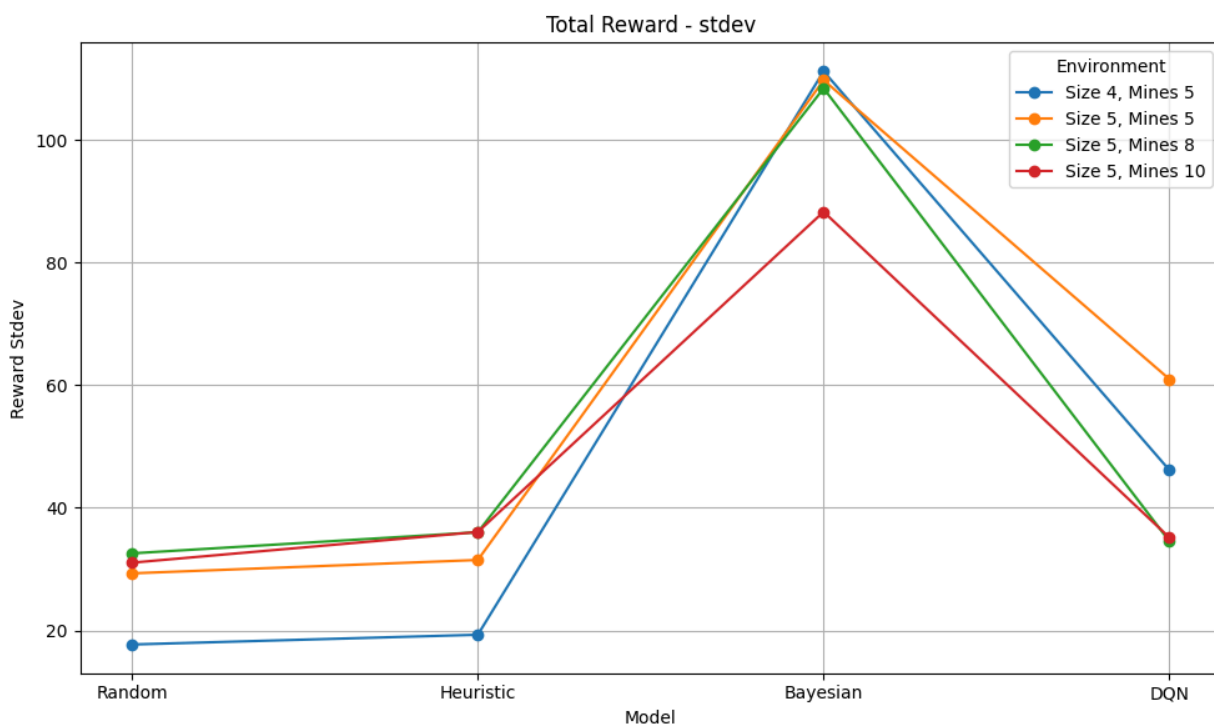
It is apparent from project outcomes that knowledge injection could have reduced the effective space that the DQN needed to search and improved learning. By narrowing the gap between random exploration and meaningful strategic behavior, the agent would have been far more likely to escape the underfitting regime and converge toward policies that reflect the actual combinatorial structure of 3D Minesweeper.

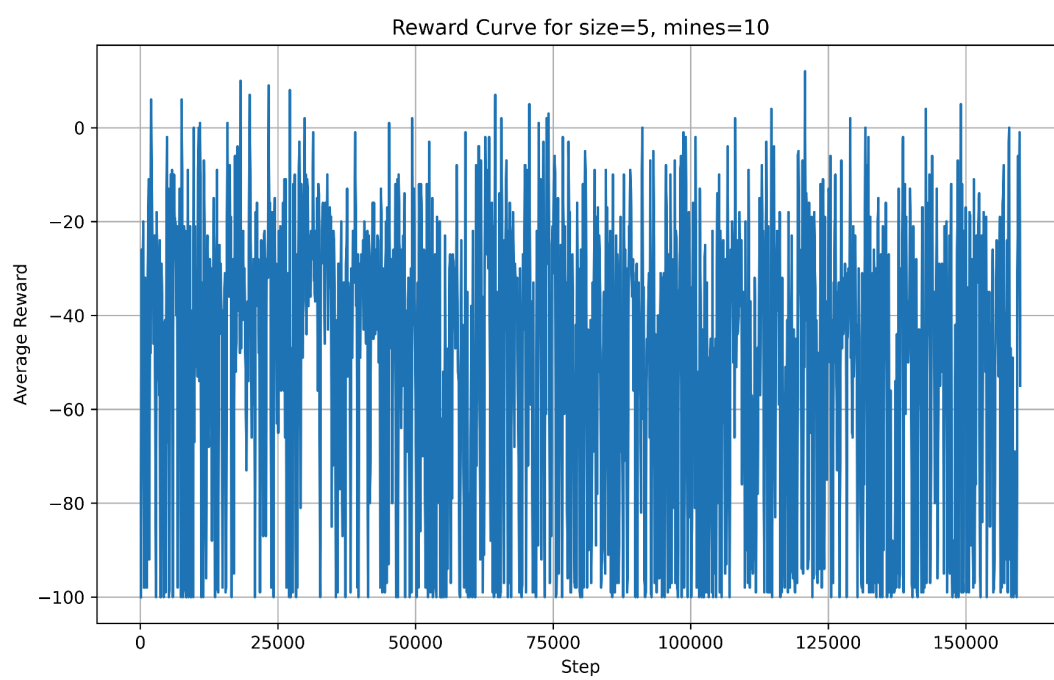
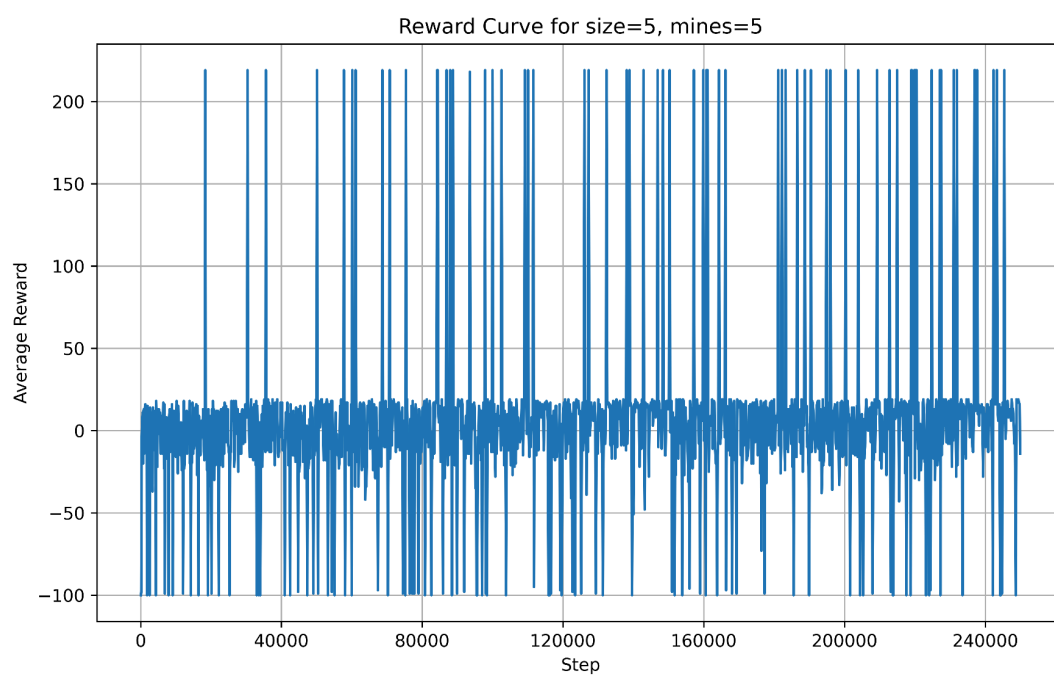
References

1. Play 3D Minesweeper, <http://egraether.com/mine3d/>
2. Play 2D Minesweeper, <https://minesweeperonline.com/>
3. 2D RL-Minesweeper Environment & Repository,
<https://github.com/markov-labs/RL-Minesweeper/blob/main/evaluation/evaluate.py>
4. Forced Guess Scenarios in Minesweeper,
https://minesweeper.fandom.com/wiki/Forced_Guess
5. CS271-Minesweeper Environment,
<https://github.com/DrNekoSenpai/CS271-Minesweeper/blob/main/backend/environment.py>
6. CS271-Minesweeper Bayesian Agent,
https://github.com/DrNekoSenpai/CS271-Minesweeper/blob/main/agents/bayesian_approximation_agent.py
7. CS271-Minesweeper DQN Agent,
https://github.com/DrNekoSenpai/CS271-Minesweeper/blob/main/agents/dueling_cnn_agent.py
8. CS271-Minesweeper DQN Training,
https://github.com/DrNekoSenpai/CS271-Minesweeper/blob/main/train_dqn.py

Appendix







Size 4, Mines 5 - Random Agent

Total Reward:

min = -100.00
max = -42.00
mean = -68.27
median = -62.00
var = 313.24
stdev = 17.70

Number of Moves:

min = 1.00
max = 34.00
mean = 6.95
median = 6.00
var = 22.63
stdev = 4.76

Win / Loss:

Wins = 0
Losses = 5000
Win rate = 0.000

Timing:

Total time = 7.99 seconds
Avg per ep = 0.0016 seconds
Episodes/sec = 625.81

Size 4, Mines 5 - Heuristic Agent

Total Reward:

min = -100.00
max = 158.00
mean = -65.28
median = -57.00
var = 372.23
stdev = 19.29

Number of Moves:

min = 1.00
max = 34.00
mean = 9.19
median = 9.00
var = 24.64
stdev = 4.96

Win / Loss:

Wins = 1
Losses = 4999
Win rate = 0.000

Timing:

Total time = 12.93 seconds
Avg per ep = 0.0026 seconds
Episodes/sec = 386.56

Size 4, Mines 5 - Bayesian Agent

Total Reward:

min = -100.00
max = 158.00
mean = 34.73
median = -42.00
var = 12372.15
stdev = 111.23

Number of Moves:

min = 1.00
max = 44.00
mean = 17.25
median = 18.00
var = 95.63
stdev = 9.78

Win / Loss:

Wins = 2211
Losses = 2789
Win rate = 0.442

Timing:

Total time = 172.38 seconds
Avg per ep = 0.0345 seconds
Episodes/sec = 29.01

Size 4, Mines 5 - DQN Agent

Total Reward:

min = -100.00
max = 158.00
mean = -58.79
median = -57.00
var = 2129.41
stdev = 46.15

Number of Moves:

min = 1.00
max = 43.00
mean = 11.21
median = 10.00
var = 65.85
stdev = 8.11

Win / Loss:

Wins = 171
Losses = 4829
Win rate = 0.034

Timing:

Total time = 192.07 seconds
Avg per ep = 0.0384 seconds
Episodes/sec = 26.03

Size 5, Mines 5 - Random Agent

Total Reward:

min = -100.00
max = 219.00
mean = -5.66
median = 3.00
var = 859.33
stdev = 29.31

Number of Moves:

min = 1.00
max = 36.00
mean = 7.79
median = 7.00
var = 22.01
stdev = 4.69

Win / Loss:

Wins = 6
Losses = 4994
Win rate = 0.001

Timing:

Total time = 17.26 seconds
Avg per ep = 0.0035 seconds
Episodes/sec = 289.71

Size 5, Mines 5 - Heuristic Agent

Total Reward:

min = -100.00

max = 219.00

mean = 2.21

median = 11.00

var = 991.64

stdev = 31.49

Number of Moves:

min = 1.00

max = 39.00

mean = 12.51

median = 11.00

var = 50.71

stdev = 7.12

Win / Loss:

Wins = 15

Losses = 4985

Win rate = 0.003

Timing:

Total time = 40.87 seconds

Avg per ep = 0.0082 seconds

Episodes/sec = 122.34

Size 5, Mines 5 - Bayesian Agent

Total Reward:

min = -100.00

max = 219.00

mean = 142.85

median = 219.00

var = 12053.78

stdev = 109.79

Number of Moves:

min = 1.00

max = 54.00

mean = 19.54

median = 19.00

var = 89.84

stdev = 9.48

Win / Loss:

Wins = 3311

Losses = 1689

Win rate = 0.662

Timing:

Total time = 216.14 seconds

Avg per ep = 0.0432 seconds

Episodes/sec = 23.13

Size 5, Mines 5 - DQN Agent

Total Reward:

min = -100.00
max = 219.00
mean = 14.51
median = 14.00
var = 3721.40
stdev = 61.00

Number of Moves:

min = 1.00
max = 48.00
mean = 14.98
median = 14.00
var = 69.48
stdev = 8.34

Win / Loss:

Wins = 311
Losses = 4689
Win rate = 0.062

Timing:

Total time = 434.81 seconds
Avg per ep = 0.0870 seconds
Episodes/sec = 11.50

Size 5, Mines 8 - Random Agent

Total Reward:

min = -100.00
max = 15.00
mean = -34.48
median = -24.00
var = 1061.11
stdev = 32.57

Number of Moves:

min = 1.00
max = 48.00
mean = 8.29
median = 7.00
var = 33.59
stdev = 5.80

Win / Loss:

Wins = 0
Losses = 5000
Win rate = 0.000

Timing:

Total time = 16.71 seconds
Avg per ep = 0.0033 seconds
Episodes/sec = 299.19

Size 5, Mines 8 - Heuristic Agent

Total Reward:

min = -100.00

max = 16.00

mean = -22.51

median = -7.00

var = 1297.77

stdev = 36.02

Number of Moves:

min = 1.00

max = 53.00

mean = 14.96

median = 15.00

var = 66.58

stdev = 8.16

Win / Loss:

Wins = 0

Losses = 5000

Win rate = 0.000

Timing:

Total time = 36.54 seconds

Avg per ep = 0.0073 seconds

Episodes/sec = 136.82

Size 5, Mines 8 - Bayesian Agent

Total Reward:

min = -100.00
max = 216.00
mean = 48.39
median = 11.00
var = 11762.78
stdev = 108.46

Number of Moves:

min = 1.00
max = 78.00
mean = 26.01
median = 27.00
var = 198.96
stdev = 14.11

Win / Loss:

Wins = 1360
Losses = 3640
Win rate = 0.272

Timing:

Total time = 400.97 seconds
Avg per ep = 0.0802 seconds
Episodes/sec = 12.47

Size 5, Mines 8 - DQN Agent

Total Reward:

min = -100.00

max = 16.00

mean = -35.93

median = -26.00

var = 1193.32

stdev = 34.54

Number of Moves:

min = 1.00

max = 43.00

mean = 9.17

median = 8.00

var = 43.83

stdev = 6.62

Win / Loss:

Wins = 0

Losses = 5000

Win rate = 0.000

Timing:

Total time = 260.74 seconds

Avg per ep = 0.0521 seconds

Episodes/sec = 19.18

Size 5, Mines 10 - Random Agent

Total Reward:

min = -100.00
max = 10.00
mean = -50.25
median = -41.00
var = 963.96
stdev = 31.05

Number of Moves:

min = 1.00
max = 42.00
mean = 8.13
median = 7.00
var = 34.15
stdev = 5.84

Win / Loss:

Wins = 0
Losses = 5000
Win rate = 0.000

Timing:

Total time = 16.42 seconds
Avg per ep = 0.0033 seconds
Episodes/sec = 304.58

Size 5, Mines 10 - Heuristic Agent

Total Reward:

min = -100.00
max = 10.00
mean = -40.25
median = -27.00
var = 1300.50
stdev = 36.06

Number of Moves:

min = 1.00
max = 52.00
mean = 14.61
median = 15.00
var = 72.13
stdev = 8.49

Win / Loss:

Wins = 0
Losses = 5000
Win rate = 0.000

Timing:

Total time = 32.93 seconds
Avg per ep = 0.0066 seconds
Episodes/sec = 151.83

Size 5, Mines 10 - Bayesian Agent

Total Reward:

min = -100.00

max = 214.00

mean = 4.62

median = 0.00

var = 7798.08

stdev = 88.31

Number of Moves:

min = 1.00

max = 85.00

mean = 26.30

median = 26.00

var = 280.07

stdev = 16.74

Win / Loss:

Wins = 629

Losses = 4371

Win rate = 0.126

Timing:

Total time = 460.98 seconds

Avg per ep = 0.0922 seconds

Episodes/sec = 10.85

Size 5, Mines 10 - DQN Agent

Total Reward:

min = -100.00
max = 12.00
mean = -52.38
median = -42.00
var = 1235.50
stdev = 35.15

Number of Moves:

min = 1.00
max = 49.00
mean = 9.32
median = 8.00
var = 47.76
stdev = 6.91

Win / Loss:

Wins = 0
Losses = 5000
Win rate = 0.000

Timing:

Total time = 283.49 seconds
Avg per ep = 0.0567 seconds
Episodes/sec = 17.64